OS Orange Problem Report

Name : Sriram K

SRN : PES1UG24CS468

SECTION : 4 'H'

Screenshot 1A:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:111:9: warning: unused variable 'dir_fd' [-Wunused-variable]
  111 |     int dir_fd = -1;
      |         ^~~~~~
object.c:161:5: warning: ignoring return value of 'write' declared with attribute 'warn_unused_result' [-Wunused-result]
  161 |     write(tmp_fd, object_buf, object_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 1B:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 2A:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ gcc -Wall -Wextra -O2 test_tree.c object.c tree.c index.c -o test_tree -lc
rypto
./test_tree
object.c: In function ‘object_write’:
object.c:111:9: warning: unused variable ‘dir_fd’ [-Wunused-variable]
  111 |     int dir_fd = -1;
      |         ^~~~~~
object.c:161:5: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  161 |     write(tmp_fd, object_buf, object_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
tree.c: In function ‘write_tree_recursive’:
tree.c:100:44: warning: unused parameter ‘id_out’ [-Wunused-parameter]
  100 |                                 ObjectID *id_out) {
      |                                 ~~~~~~~~~~~^~~~~~
tree.c: At top level:
tree.c:98:12: warning: ‘write_tree_recursive’ defined but not used [-Wunused-function]
   98 | static int write_tree_recursive(IndexEntry *entries, int count,
      |            ^~~~~~~~~~~~~~~~~~~~
index.c: In function ‘index_load’:
index.c:134:9: warning: ‘__builtin_strncpy’ output may be truncated copying 511 bytes from a string of length 511 [-Wstringop-trunca
tion]
  134 |         strncpy(e->path, path, sizeof(e->path) - 1);
      |         ^
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 2B:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ find .pes/objects -type f
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 3A:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ make clean && make pes
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ ./pes add file1.txt file2.txt
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  pes.h
  untracked:  index.c
  untracked:  commit.h
  untracked:  README.md
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  index.h
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  Makefile

sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 3B:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776435639 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776435647 6 file2.txt
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 4A:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ ./pes init
echo "First file" > hello.txt
./pes add hello.txt
./pes commit -m "My first commit"
Initialized empty PES repository in .pes/
Committed: d509f8e31453... My first commit
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: a0b17d1242ff... Add world
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: 3b25fbf0b54d... Add farewell
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ ./pes log
commit 3b25fbf0b54d527c867afd31edda4ade475f750669239c771b078aa470d03784
Author: PES User <pes@localhost>
Date:   1776440742

    Add farewell

commit a0b17d1242ffc25bae6538240a4dfea21ef37f1c2454f018bf40ebc852fbb78d
Author: PES User <pes@localhost>
Date:   1776440733

    Add worldell

commit d509f8e31453db47043819af13d9df370d5b72ab8f4ab2a329d1d9122ad0dd84
Author: PES User <pes@localhost>
Date:   1776440670

    My first commitirst commQ

sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 4B:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/3b/25fbf0b54d527c867afd31edda4ade475f750669239c771b078aa470d03784
.pes/objects/93/10136230d78dc102bd924778ece46a160633d56bd7ffb1db6970f7c8301e49
.pes/objects/a0/b17d1242ffc25bae6538240a4dfea21ef37f1c2454f018bf40ebc852fbb78d
.pes/objects/a9/44098a53d9220a295cf2677ba68606610f4c803e36aa743979e83842c1deb2
.pes/objects/c6/9b183892026a777613a3666d17e399b278eadc3f6faa7a97c9a0db7ab2f85a
.pes/objects/d3/e162d8f05113244ad2db6c3d314413222e36444936bed20eef99af1782497a
.pes/objects/d5/09f8e31453db47043819af13d9df370d5b72ab8f4ab2a329d1d9122ad0dd84
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f4/841e200f72b3179434d9bd726b4a9ac6fdf8b275cb98f768ce599ffc271cee
.pes/refs/heads/main
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Screenshot 4C:

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ cat .pes/refs/heads/main
3b25fbf0b54d527c867afd31edda4ade475f750669239c771b078aa470d03784
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$
```

Final Screenshot :

```
sriram@sriram-VirtualBox:~/pes-vcs/PES1UG24CS468-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 2fc9baab5e28... Initial commit

Log after first commit:
commit 2fc9baab5e28733874acd0c3f7ef8810d4552e308a3c24f5b2f8208bf7dcb005
Author: PES User <pes@localhost>
Date:   1776440949

    Initial commit
```

Phase 5: Branching and Checkout

Q5.1: Implementation of pes checkout <branch>

- **Files in .pes/ to change:** The .pes/HEAD file must be updated to point to the new branch reference (e.g., ref: refs/heads/<branch>).
- **Working Directory Actions:** The current files in the working directory must be deleted or overwritten to match the tree object of the target branch.
- **Complexity Factors:** Complexity arises from managing "dirty" files (unsaved work), ensuring that the .pes/index is updated to match the new state, and handling files that exist in the current branch but not in the target branch to avoid leaving "ghost" files behind.

Q5.2: Detecting a "Dirty Working Directory" Conflict

- **Detection Logic:** To detect a conflict, the system must compare the current file's metadata (mtime and size) in the working directory against the metadata stored in the .pes/index.
- **Conflict Logic:** If the index entry for a file is different from the version in the actual working directory (unstaged changes), and that same file in the target branch's tree has a different hash, a conflict exists.
- **Security:** By checking if the working directory hash matches the current index hash before switching, the system ensures uncommitted work is not overwritten.

Q5.3: Detached HEAD State

- **Commit Behavior:** In a "Detached HEAD" state, new commits are created normally, and the HEAD file is updated to the new commit hash. However, since no branch reference (like main) points to these commits, they become "orphaned" once the user switches back to a named branch.
- **Recovery:** A user can recover these commits by looking at the reflog (a history of where HEAD has been) to find the hash of the orphaned commit and then creating a new branch at that hash (e.g., pes branch recovery-branch <hash>).

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Mark-and-Sweep Algorithm for GC

- **Algorithm Description:** Use a "Mark-and-Sweep" approach. Start at all known "roots" (HEAD, all branch refs in refs/heads/, and tags) and recursively traverse the commit objects to find their tree hashes, then the blobs within those trees.
- **Data Structure:** A **Bloom Filter** or a **Hash Set** would be most efficient to track the "reachable" hashes and prevent redundant visits to shared objects.
- **Estimate:** For 50 branches and 100,000 commits, the algorithm would visit 100,000 commit objects plus their unique tree and blob objects. Since many commits share trees, the number of objects visited is typically proportional to the total number of unique files and directory states ever recorded in the history.

Q6.2: GC Race Conditions and Safety

- **The Danger:** If GC runs while a commit is in progress, the GC might identify a new blob as "unreachable" (because the commit object pointing to it hasn't been written yet) and delete it.
- **Race Condition Example:** A pes add operation writes a blob to the object store. Before the pes commit finishes writing the commit object that references that blob, the GC deletes the "unreferenced" blob. The resulting commit will be broken and point to a missing object.
- **Git's Solution:** Git avoids this by using a **grace period** (usually 2 weeks). GC will only delete unreachable objects that are older than a certain timestamp, ensuring that any objects currently being written by active processes are safe.